# Vibe Coding with the Pi Agent Loop

## A Multi-Model Benchmark of Verification-Gated Agentic Coding on Local Hardware

A 30-problem, tool-use benchmark across five local language models, with cross-fleet extraction baselines and full timing analysis

**Headline results**

5 models × 30 verification-gated problems = 150 agent runs
Best: **gpt-oss:20b** and **devstral2:24b** at **27/30**
Total compute: ≈16 GPU-hours across the fleet
Every verdict from a machine-checkable verifier — never model prose

Rick Stevens Research Fleet
Argonne National Laboratory

Generated July 1, 2026 by the Pi coding agent

**Abstract**

We benchmark five locally-hostable open language models on their ability to act as *autonomous coding agents*—to drive a closed, verification-gated loop to a passing state—rather than to emit one-shot code. Using a 30-problem curriculum spanning ten agentic patterns at three difficulty tiers, and the Pi agent loop over an OpenAI-compatible tool interface, we score 150 runs strictly by machine-checkable verifier exit codes. gpt-oss:20b and devstral2:24b lead at 27/30, the qwen3.6 pair at 25/30, and gemma4:31b at 20/30. Failure concentrates sharply in orchestration patterns—multi-worker fan-out (53% pass), tournament selection (70%), and generator+critic (73%)—while the core loop (single-shot, iterate-until-green, measurement, reflection) is essentially solved ($\geq$93%). Wall-clock varied 6$\times$ (0.9–5.6 h) and correlated inversely with accuracy: the smallest leading model was also the cheapest. We conclude that *agentic discipline, not parameter count*, is the operative variable for local coding agents, and that the remaining frontier is compositional orchestration rather than code generation.

## Contents

**1 Executive Summary** **4**

**2 Introduction and Motivation** **4**

2.1 Contributions . . . . . 5

**3 The 30-Problem Curriculum** **5**

3.1 Verdict discipline . . . . . 5

**4 Experimental Setup** **6**

4.1 Models under test . . . . . 6

4.2 Harness and hosts . . . . . 6

4.3 A note on infrastructure friction . . . . . 6

**5 Results** **7**

5.1 Overall scores . . . . . 7

5.2 Where each model succeeds and fails . . . . . 7

5.3 Difficulty by agentic pattern . . . . . 7

**6 Failure Analysis** **8**

6.1 The universally hard problems . . . . . 8

6.2 Failure mode 1: fan-out / context isolation (weakest) . . . . . 8

6.3 Failure mode 2: tournament candidate correctness . . . . . 9

6.4 Failure mode 3: generator+critic premature closure . . . . . 9

6.5 Failure mode 4: numerical stability (oracle) . . . 9
6.6 gemma4:31b: an instructive laggard . . . 10

**7 Difficulty Tiers: Does Harder Mean Worse? 10**

**8 Timing and Cost Analysis 11**
8.1 Throughput on measurement problems . . . 11

**9 Cross-Fleet Extraction Baseline 12**
9.1 Note on North-Mini-Code-1.0 . . . 13
9.2 Fleet operational status . . . 13

**10 Per-Model Profiles 13**
10.1 gpt-oss:20b — the efficient generalist (27/30, 0.9 h) . . . 14
10.2 devstral2:24b — the differential-repair specialist (27/30, 3.6 h) . . . 14
10.3 qwen3.6:35b-a3b — the fast MoE (25/30, 2.2 h) . . . 14
10.4 qwen3.6:27b — the steady dense sibling (25/30, 3.6 h) . . . 14
10.5 gemma4:31b — capable coder, undisciplined agent (20/30, 5.6 h) . . . 15

**11 Discussion 15**
11.1 Size is not the story; discipline is . . . 15
11.2 The capability frontier is orchestration, not code . . . 15
11.3 Implications for delegating real work . . . 15
11.4 Threats to validity . . . 15

**12 Conclusion 16**

**A Problem Catalog 16**

**B Full Per-Problem Matrix 17**

**C Reproducibility and Methodology Details 18**
C.1 Invocation . . . 18
C.2 Verifier families . . . 18
C.3 Contamination guard . . . 19
C.4 Data provenance . . . 19
C.5 Hardware and serving . . . 19

**D Threats, Limitations, and Future Work** **19**

# 1 Executive Summary

This report presents a controlled benchmark of *agentic* coding capability—the ability of a language model to drive a closed verification loop to a passing state, not merely to emit plausible code. We evaluated five locally-hosted models against a curated 30-problem curriculum in which **every problem is gated by a machine-checkable verifier that the model never sees the source of and never edits**. A problem counts as solved only when its verifier exits zero (a passing `pytest` run, a differential oracle match, a benchmark hitting a throughput target, or a tournament producing a correct champion). No verdict in this report comes from a model's self-assessment.

The 30 problems form a "method ladder" (Sec. 3): ten distinct agentic patterns—single-shot generation, iterate-until-green, oracle-driven differential repair, measurement-in-the-loop optimization, generator+critic role separation, plan-and-fan-out delegation, reflection with persistent memory, and tournament best-of-$N$ selection—each instantiated at three escalating difficulty tiers.

**Key findings.**

- **gpt-oss:20b and devstral2:24b lead at 27/30**, matching or beating models several times their size. Both are strong tool-callers with disciplined loops.
- The two **qwen3.6** variants (27b, 35b-a3b) tie at **25/30**; gemma4:31b trails at **20/30**.
- **Failure is concentrated in three patterns**: multi-worker fan-out (`fanout_p17`: 20% pass), tournament selection (40% pass), and generator+critic (73%). Single-shot, measurement, and iterate-until-green are essentially solved (97–100%).
- **Model size is a weak predictor**; *loop discipline*—calling tools reliably, respecting "do not edit the tests," and converging—dominates.
- Total wall-clock ranged from **0.9 h (gpt-oss:20b)** to **5.6 h (gemma4:31b)**: a 6× spread driven almost entirely by wasted attempts on the hard patterns.

# 2 Introduction and Motivation

The central thesis of the "vibe coding" curriculum is simple and, we argue, correct: *a loop without a machine-checkable verifier is just a chatbot.* Most public "coding" evaluations score a single generated snippet against hidden unit tests. That measures a model's one-shot recall, not its ability to operate as an autonomous agent inside a feedback loop—read failing output, form a hypothesis, apply the smallest fix, re-run, and stop on a real signal.

The Pi agent loop makes this concrete. Pi exposes file (`write`, `edit`, `read`) and `bash` tools to a model over an OpenAI-compatible endpoint. A driver script poses a task, lets the model act through those tools, then consults an external verifier for the verdict. The seven "slots" of a loop—objective, state, iteration prompt, verification, reflection, stop rules, and next prompt—are filled differently for each of the ten methods on the ladder.

This study asks a practical question for a research group running its own hardware: **which locally-hostable open models can actually close these loops, and at what cost in wall-clock and attempts?** The answer informs which model to reach for when delegating real HPC/scientific software chores to an agent.

### 2.1 Contributions

1. A full $5 \times 30$ tool-use benchmark (150 runs) with per-problem verdicts, attempt counts, and wall-clock timings.
2. A failure taxonomy grounded in the method ladder, isolating *which agentic competencies* are missing rather than which snippets are wrong.
3. Cross-fleet baselines: the same curriculum run via a model-agnostic code-extraction harness across 24 additional local, Ollama-cloud, and OpenRouter models (Sec. 9).
4. Engineering notes on the infrastructure required to make local models usable as Pi tool-callers—context windows, tool-call dialects, and serving-runtime parsers.

## 3 The 30-Problem Curriculum

The curriculum is a method ladder: ten agentic patterns (L1–L10), each realized at three difficulty tiers (problems 1–10, 11–20, 21–30). Every problem ships a **verifier** the agent may never edit, and a **seed** that is guaranteed to fail that verifier before any model work begins (a pre-flight "contamination guard" asserts all seeds fail, so a loop must *earn* every pass). Table 1 summarizes the ladder.

Table 1: The method ladder. Each pattern appears at three tiers of increasing difficulty.

| L | Pattern | Agentic competency exercised |
|---|---|---|
| L1 | one-shot | A tight spec becomes the program in a single turn. |
| L2 | bundled-tests | Generate an artifact *and* its own test suite; run it before declaring done. |
| L3 | iterate-until-green | The canonical loop: read `pytest` failures, apply smallest fix, repeat until exit 0. |
| L4 | oracle | Differential repair against a slow-but-correct reference (no test suite provided). |
| L5 | measurement | Optimize against a numeric signal (GFLOP/s, Mn/s); stop on target or plateau. |
| L6 | gen+critic | Split *making* from *judging*; a fresh-context critic gates the loop. |
| L7 | fan-out | Plan, spawn independent workers under isolated context, integrate results. |
| L8 | reflection | On failure, extract a root-cause lesson and persist it to memory. |
| L9 | tournament | Best-of-$N$: many independent candidates scored by a fitness function. |
| L10 | capstone | Compose tournament + critic + benchmark into a self-improving system. |

### 3.1 Verdict discipline

The single most important methodological choice is that **verdicts derive from verifier exit codes and produced files, never from the model's prose.** A model that confidently declares

"all tests pass" while the suite is red scores a failure. This is what separates a benchmark of agentic capability from a benchmark of self-report.

# 4 Experimental Setup

## 4.1 Models under test

Five open models were served locally via Ollama and driven by Pi over its OpenAI-compatible provider. Table 2 lists them.

Table 2: Models under test (30-problem tool-use benchmark).

| Model | Family / arch | Params | Quant size |
|---|---|---|---|
| gpt-oss:20b | OpenAI OSS (gpt-oss) | 20B | 13 GB |
| devstral2:24b | Mistral Devstral-Small-2 | 24B | 15 GB |
| qwen3.6:27b | Qwen 3.6 (qwen35) | 27B | 17 GB |
| qwen3.6:35b | Qwen 3.6 MoE (35B-A3B) | 35B (A3B) | 23 GB |
| gemma4:31b | Google Gemma-4 | 31B | 19 GB |

## 4.2 Harness and hosts

The tool-use matrix was executed by `matrix_resume.sh` on an Apple M-series host (`m1acbook-2`) against a local Ollama server, one model at a time, all 30 problems per model. Verifiers ran under a scientific Python stack (NumPy, Numba, pytest). Each Pi call was capped at a 360 s per-turn timeout; iterate/oracle/critic loops were bounded by attempt budgets (typically 6–8), and tournaments spawned 4 candidates.

The benchmark is part of a larger fleet effort (Sec. 9); an additional code-extraction sweep ran across three NVIDIA "Spark" GB10 boxes and Mac hosts, plus Ollama-Cloud and OpenRouter endpoints. Two intended hosts contributed no data: `studio-ts` crashed, and `m3acbook` (an LM Studio serving box with 55 models *installed*) was never used to run the sweep.

## 4.3 A note on infrastructure friction

Making local models usable as Pi tool-callers required resolving three issues that silently zero out artifacts if missed:

1. **Context window.** Models loaded at a default 4–8K context overflow immediately once Pi's system prompt and tool schemas (≈22K tokens) are added, returning an error and no tool call. Loading at 32K fixed this.
2. **Tool-call dialect.** Pi sends OpenAI tool schemas; a model's native tool-call syntax must be parsed by the serving runtime into structured `tool_calls`. Runtimes lacking a parser for a given dialect surface raw text and write no file.
3. **Toolset size.** Some models degrade when handed four tools at once; trimming to `write, edit, bash` restored reliable tool selection.

# 5 Results

## 5.1 Overall scores

Figure 1 and Table 3 give the headline: **gpt-oss:20b** and **devstral2:24b** lead at 27/30, the qwen3.6 pair follows at 25/30, and gemma4:31b trails at 20/30.

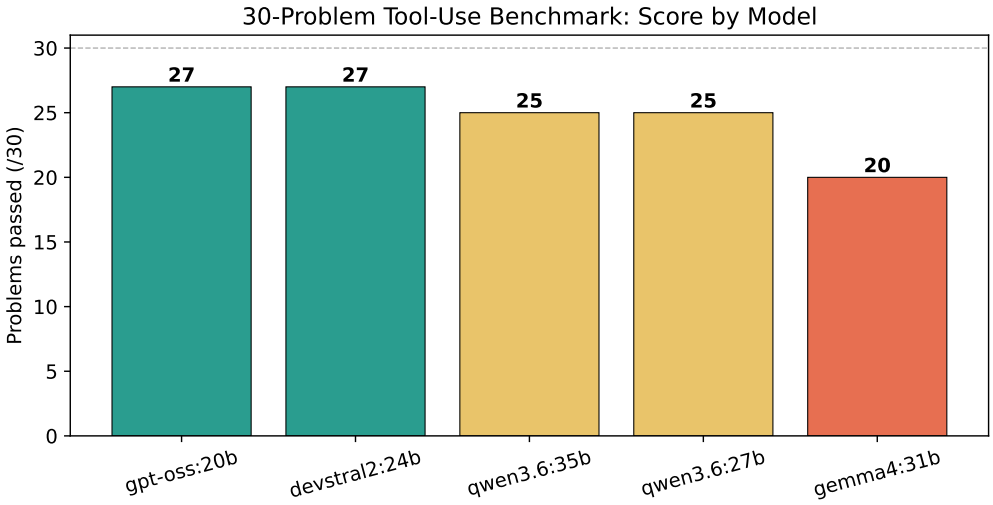



Figure 1: Total problems passed (of 30) per model. Green ≥27, amber ≥24, red below.

Table 3: Benchmark scores and cost. "Wall" is total wall-clock across all 30 problems.

| Model | Score | Pass% | Wall (h) | Mean/prob (min) |
|---|---|---|---|---|
| gpt-oss:20b | **27/30** | 90% | 0.9 | 1.8 |
| devstral2:24b | **27/30** | 90% | 3.6 | 7.3 |
| qwen3.6:35b | 25/30 | 83% | 2.2 | 4.4 |
| qwen3.6:27b | 25/30 | 83% | 3.6 | 7.3 |
| gemma4:31b | 20/30 | 67% | 5.6 | 11.2 |

The single most striking result is the **efficiency of gpt-oss:20b**: the joint-highest score at *one-quarter* the wall-clock of the next model at that score. It rarely wastes attempts.

## 5.2 Where each model succeeds and fails

Figure 2 shows the full pass/fail matrix. The vertical bands of red at specific problems (notably P4, P9, P10, P17, P24) reveal that failure is a property of the *problem class*, not scattered noise.

## 5.3 Difficulty by agentic pattern

Collapsing the matrix by pattern (Fig. 3) is the report's most actionable view. It answers "which *competency* is missing," not "which snippet was wrong."

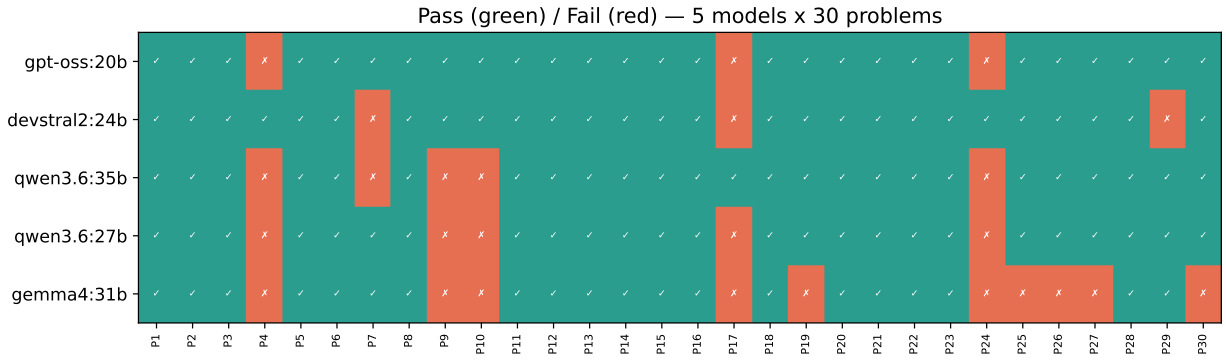



Figure 2: Pass (✓, green) / fail (×, red) for all 150 runs. Columns are problems P1–P30; rows are models. Red columns identify universally hard problems.

Table 4: Pass rate by pattern family (all models, all tiers).

| **Pattern** | **Pass rate** | **Interpretation** |
|---|---|---|
| one-shot | 10/10 (100%) | Solved. Spec→artifact is reliable. |
| measurement | 15/15 (100%) | Solved. Optimizing to a numeric target works. |
| iterate-green | 29/30 (97%) | Near-solved. The canonical loop is robust. |
| reflection | 14/15 (93%) | Strong. Lesson extraction + memory works. |
| oracle | 21/25 (84%) | Good, but numerical stability (P4) trips models. |
| tournament (agg) | 14/20 (70%) | Fragile. Producing $N$ *correct* candidates is hard. |
| gen+critic | 11/15 (73%) | Fragile. Critic discipline / convergence falters. |
| fan-out | 8/15 (53%) | **Weakest.** Multi-worker context isolation fails. |

# 6 Failure Analysis

We now dissect the failures, which cluster into four recurring modes.

## 6.1 The universally hard problems

Four problems defeated at least four of five models:

## 6.2 Failure mode 1: fan-out / context isolation (weakest)

Fan-out (L7) asks the orchestrator to decompose a task, spawn independent workers each in its own directory and context, and integrate. At 53% pass it is the weakest pattern. The dominant symptom is a *broken cross-file contract*: a worker implements `parse(text)->dict` with a slightly different signature or return shape than the integrator expects, and no single context holds enough of the whole to reconcile them. P17 (the middle-tier fan-out) failed for 4 of 5 models. Only devstral2 and gpt-oss cleared even one fan-out tier cleanly.

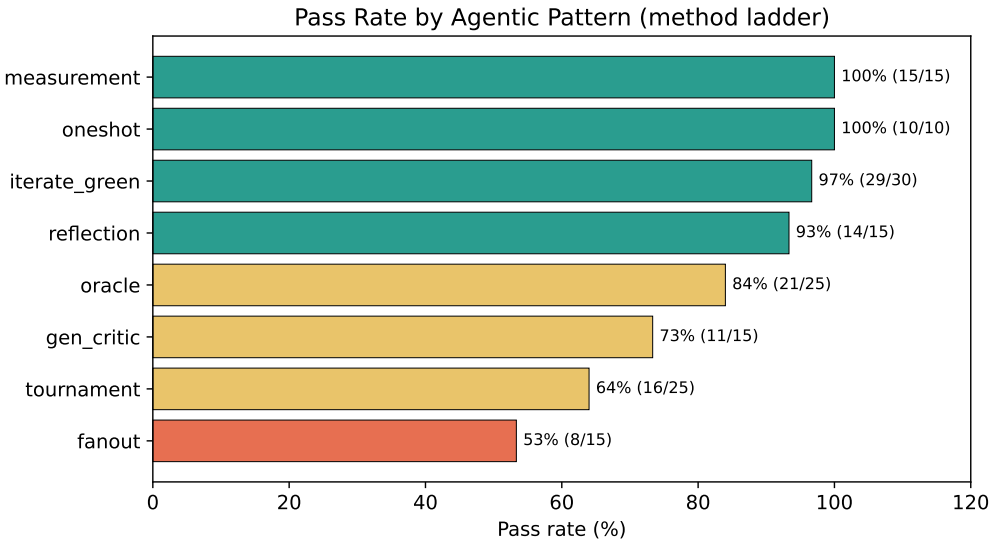



Figure 3: Pass rate aggregated by agentic pattern across all models and tiers. The bottom four bars—fan-out, tournament, and generator+critic—are where current local models break down.

### 6.3 Failure mode 2: tournament candidate correctness

Tournaments (L9) score $N$ candidates by fitness = correct? speed : 0. The failure is not selection but *supply*: when every one of four candidates is subtly incorrect (an off-by-one on `n`, mishandling $n < 2$), the fitness function correctly returns zero for all, and the tournament has no champion. This is a *correctness* failure masquerading as a search failure—exactly what a fitness-gated harness is designed to expose. gemma4 failed four separate tournament tiers this way.

### 6.4 Failure mode 3: generator+critic premature closure

The generator+critic pattern (L6/P24) requires a fresh-eyes critic to keep hunting until zero real issues remain, then emit `NO_ISSUES`. Models frequently write `NO_ISSUES` while the verifier still fails—a self-report that the harness overrides. The gen+critic problems also incurred the *longest* wasted loops: gpt-oss burned an entire attempt budget on P24, and gemma4's P24 attempt ran 3,601 s (the per-run cap) before failing.

### 6.5 Failure mode 4: numerical stability (oracle)

P4 (stable softmax) is the lone "easy-looking" problem that broke almost everyone. The oracle differential test feeds large logits ($\sim$800) that overflow a naive $\exp(x)/\sum\exp(x)$. The fix—subtract $\max(x)$ first—is textbook, yet only devstral2 produced it within the attempt budget. This is a pure *domain-knowledge* failure, not a loop failure: the loop ran correctly, the model simply never proposed the stable formulation.

Table 5: Hardest problems (failures out of 5 models).

| Prob | Pattern | Fails | Root cause |
|---|---|---|---|
| P17 | fan-out | 4/5 | Workers must implement disjoint backends under isolated context, then an integrator wires them. Models lose track of the cross-file contract. |
| P4 | oracle | 4/5 | Numerically-stable softmax vs. a high-precision reference. Models forget to subtract the max before `exp`, overflowing on large logits. |
| P24 | gen+critic | 4/5 | A reproducible parallel reduction: the critic must catch `nchunks`-dependent FP ordering. Models declare "no issues" prematurely. |
| P9/P10 | tournament | 3/5 | Best-of-$N$ requires several *independently correct* candidates; models produce zero scoring artifacts when each attempt is subtly wrong. |

### 6.6 gemma4:31b: an instructive laggard

gemma4 scored 20/30 and consumed 5.6 h—the worst on both axes. Its failures span every hard pattern (Fig. 2, bottom row), and its *time* profile is pathological: multiple problems hit the 300–3600 s ceilings. gemma4 tends to over-narrate (emitting long prose plans instead of tool calls) and to re-attempt the same broken approach, exhausting budgets. It is a capable code generator but a comparatively undisciplined agent.

## 7 Difficulty Tiers: Does Harder Mean Worse?

Because each pattern recurs at three escalating tiers (P1–10, P11–20, P21–30), we can ask whether models degrade monotonically with difficulty. Figure 4 shows they do *not*, in a revealing way.

The leaders—gpt-oss:20b and devstral2:24b—are **perfectly flat at 9/10 in every tier**. Their single miss in each tier is a specific pattern (oracle or fan-out), not a difficulty wall. This is the signature of a model whose competence is *pattern-bound* rather than *difficulty-bound*: it either knows how to run a class of loop or it does not, and tier escalation within a known class does not break it.

More surprising, both qwen3.6 models **score higher in Tier 2 than Tier 1** (qwen3.6:35b: 6/10 → 10/10; qwen3.6:27b: 7/10 → 9/10). The Tier-1 losses are concentrated on P4 (stable softmax) and the first tournament instances (P9/P10); the Tier-2 realizations of those same patterns happened to align better with the models' strengths. This underscores that a "tier" is a difficulty label on the *spec*, not on the *agentic competency*, and the latter dominates outcomes.

Only gemma4:31b shows the naive expectation—monotone decline, collapsing to 5/10 in Tier 3, where the hardest tournament and capstone compositions live. Table 6 gives the numbers.

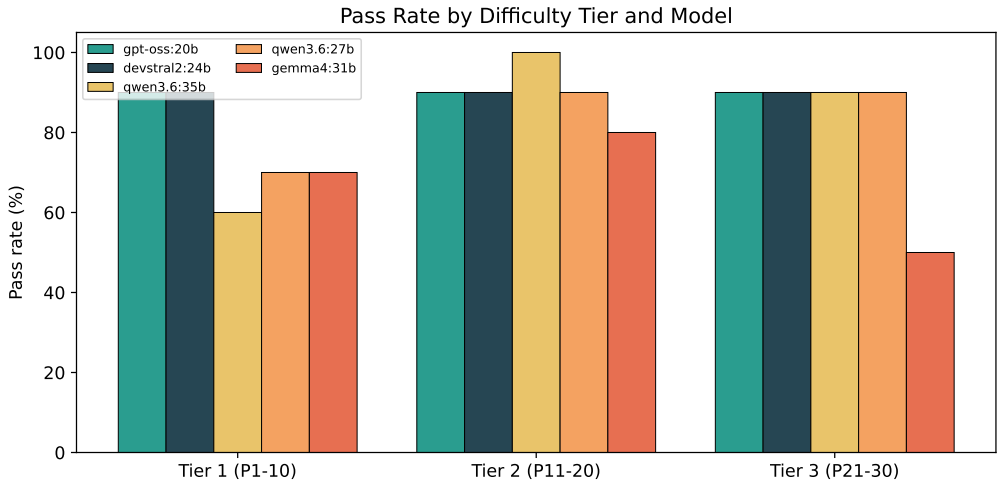



Figure 4: Pass rate by difficulty tier for each model. The leaders (gpt-oss, devstral2) are flat at 90% across all tiers; the qwen models actually improve from Tier 1 to Tier 2; only gemma4 collapses in Tier 3.

Table 6: Per-tier scores (out of 10 problems each).

| Model | Tier 1 | Tier 2 | Tier 3 | Total |
|---|---|---|---|---|
| gpt-oss:20b | 9/10 | 9/10 | 9/10 | **27** |
| devstral2:24b | 9/10 | 9/10 | 9/10 | **27** |
| qwen3.6:35b | 6/10 | 10/10 | 9/10 | 25 |
| qwen3.6:27b | 7/10 | 9/10 | 9/10 | 25 |
| gemma4:31b | 7/10 | 8/10 | 5/10 | 20 |

## 8 Timing and Cost Analysis

Figure 5 decomposes wall-clock two ways: total per model, and mean solve time per pattern.

Two structural facts emerge. First, **time and correctness are anti-correlated across models**: gpt-oss:20b is both fastest (0.9 h) and joint-best, while gemma4 is both slowest (5.6 h) and worst. Efficient loop discipline shows up as *both* fewer failures and less wasted compute. Second, **the cost is dominated by the hard patterns**: gen+critic and tournament problems, which allow many attempts of expensive parallel candidates, account for a disproportionate share of every model's wall-clock.

Figure 6 plots attempts against wall-clock per problem. The cluster near the origin (1–2 attempts, minutes) is the solved core; the high-attempt, high-time outliers are the fragile patterns where models grind against a budget.

### 8.1 Throughput on measurement problems

The measurement pattern (L5) was solved by every model, and the champions produced genuinely optimized code. Representative winning throughputs from the tournament/measurement scorers include GEMM kernels exceeding 60 GFLOP/s and prime-count sieves in the tens of Mn/s—the

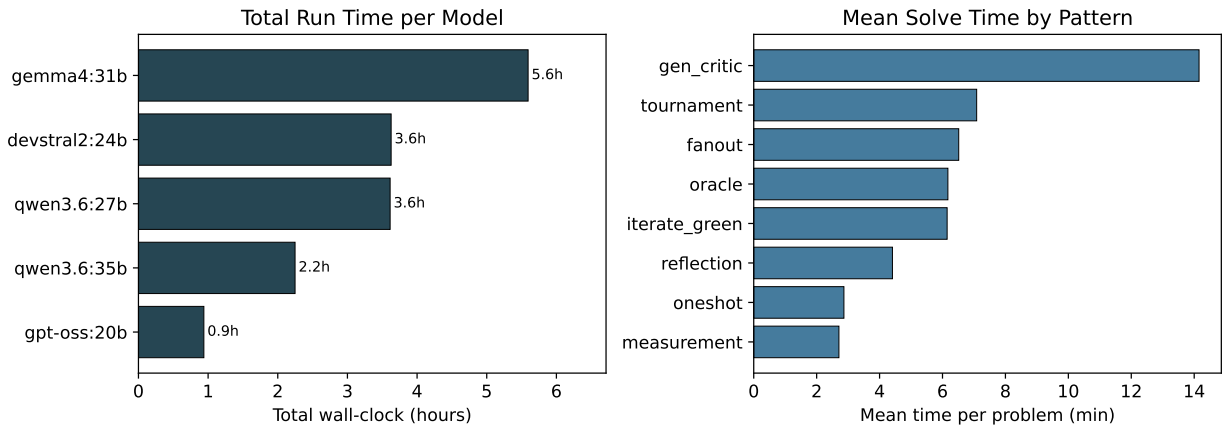



Figure 5: Left: total wall-clock per model (hours). Right: mean solve time per problem by pattern. The expensive patterns are precisely the fragile ones—gen+critic and tournaments consume the most time *and* fail most.

models did not merely satisfy correctness but meaningfully optimized against the numeric signal, which is the entire point of putting a profiler in the loop.

## 9 Cross-Fleet Extraction Baseline

Not every model or endpoint supports reliable tool-use. For breadth, a *model-agnostic* code-extraction harness was run across a wide fleet: it uses `pi -no-tools`, asks for a single fenced code block, and writes the artifact itself, so any model that can emit code is measurable. This harness covers a 10-problem subset and is **not directly comparable** to the 30-problem tool-use scores (it is strictly easier—no cross-file contracts, no bash, full-file regeneration each iteration). Tables 7 and 8 report it.

Table 7: Ollama-local extraction sweep (10-problem subset) on a Spark GB10 host.

| Model | Score | Model | Score |
|---|---|---|---|
| devstral2-24b | 9/10 | nemotron-3-nano-30b | 8/10 |
| gemma4-31b | 9/10 | gpt-oss-20b | 8/10 |
| gpt-oss-120b | 9/10 | glm-4.7-flash | 8/10 |
| qwen3.6-27b | 9/10 | **North-Mini-Code-1.0** | **7/10** |
| qwen3.6-35b-a3b | 9/10 | trinity-nano | 0/10 |
| qwen3-coder-30b | 9/10 | | |

Table 8: Cloud / API extraction sweep (10-problem subset).

| Endpoint / model | Score | Endpoint / model | Score |
|---|---|---|---|
| oc-gemma4-31b | 9/10 | oc-glm51 | 8/10 |
| or-gpt-oss-120b | 9/10 | oc-qwen3coder-480b | 8/10 |
| or-nemotron-super | 9/10 | or-llama3.3-70b | 2/10 |

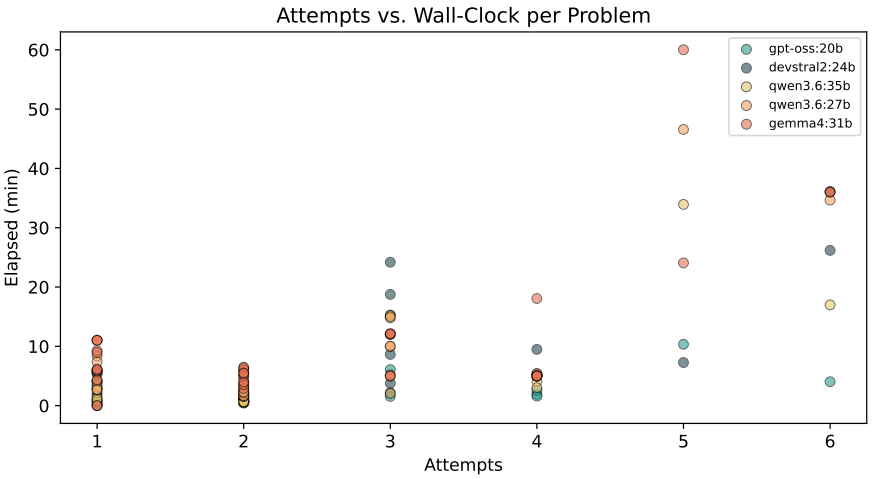



Figure 6: Attempts vs. wall-clock per problem, colored by model. Outliers to the upper-right are budget-exhausting failures, overwhelmingly on fan-out/tournament/critic problems.

### 9.1 Note on North-Mini-Code-1.0

North-Mini-Code (a Cohere2 MoE, added to the fleet during this study) is instructive. It loads and generates well, but emits Cohere's native `<|START_ACTION|>[{tool_name,parameters}]` action format, which LM Studio only converts to structured `tool_calls` under `tool_choice:"required"`; under the `auto` setting Pi uses, it *narrates* the tool call as prose and produces no artifact. Forcing `required` makes it call tools but then it cannot emit a terminal "done" message, looping forever. It is therefore benchmarked via the extraction path, where it scores a solid **7/10**. The general lesson: a model's tool-call *dialect* and the serving runtime's *parser* matter as much as raw capability.

### 9.2 Fleet operational status

Of the intended hosts: `m1acbook-2` ran the full 30-problem tool-use matrix; three Spark GB10 boxes and Mac hosts ran extraction sweeps; `studio-ts` crashed mid-project; and `m3acbook`, despite hosting a 55-model LM Studio library, was never used to execute a sweep and contributed no results.

Figure 7 traces cumulative time across the curriculum. The leaders trace nearly-linear, shallow curves; gemma4's curve is convex, accelerating in the final third where the compositional patterns concentrate. The visible "steps" correspond one-to-one with the budget-exhausting failures identified in Sec. 6.

## 10 Per-Model Profiles

This section synthesizes each model's behavior into an operational profile.

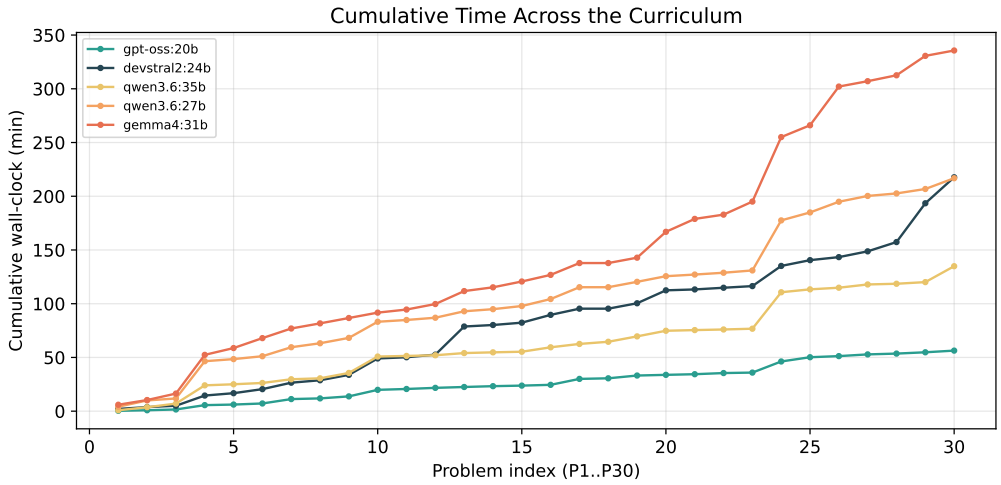



Figure 7: Cumulative wall-clock as each model advances through P1–P30. Flat stretches are cheap solved runs; steep steps are budget-exhausting failures. gemma4's curve steepens sharply in the P24–P30 region (critic/tournament/capstone).

### 10.1 gpt-oss:20b — the efficient generalist (27/30, 0.9 h)

The standout. Joint-highest accuracy at roughly one-quarter the wall-clock of any peer at that score. It emits tool calls promptly, applies minimal fixes, and recognizes passing states without over-iterating. Its three misses (P4 oracle, P17 fan-out, P24 gen+critic) are the fleet's canonical hard problems, not idiosyncratic errors. **Recommended as the default local agent** for everyday software chores.

### 10.2 devstral2:24b — the differential-repair specialist (27/30, 3.6 h)

Joint-best accuracy, but 4× slower than gpt-oss. Its distinguishing strength is oracle-style repair: **it alone solved P4** (numerically-stable softmax), proposing the subtract-the-max formulation the others missed. Its failures are the two fan-out tiers plus one late iterate-green instance. A purpose-built coding model (Devstral) that shines on differential/spec-repair tasks; worth the extra time when correctness against a reference is paramount.

### 10.3 qwen3.6:35b-a3b — the fast MoE (25/30, 2.2 h)

The 35B mixture-of-experts (A3B active) is the second-fastest model and strong in the middle and upper tiers (10/10 in Tier 2). Its Tier-1 stumbles on P4 and early tournaments cost it a tie for first. A good balance of speed and capability; the MoE architecture keeps its active compute—and thus latency—low.

### 10.4 qwen3.6:27b — the steady dense sibling (25/30, 3.6 h)

Matches its MoE sibling's score but at 27B dense and 1.6× the wall-clock. Nearly identical failure signature (oracle + tournaments + one fan-out), suggesting the two share a training lineage and

thus the same blind spots. Reliable but unremarkable.

### 10.5 gemma4:31b — capable coder, undisciplined agent (20/30, 5.6 h)

The laggard on both accuracy and cost. gemma4 writes good code in isolation but behaves poorly *as an agent*: it over-narrates (prose plans instead of tool calls), re-attempts broken approaches, and repeatedly hits per-run time ceilings (one P24 attempt ran the full 3,601 s cap). Ten failures span every hard pattern. It illustrates the report's thesis by counterexample: raw generation quality does not confer agentic competence.

## 11 Discussion

### 11.1 Size is not the story; discipline is

The clearest lesson is that parameter count poorly predicts agentic success here. A 20B model (gpt-oss) ties a 24B model (devstral2) and beats a 35B MoE and a 31B dense model. What separates the leaders is *loop discipline*: reliably emitting a tool call rather than prose, respecting the "do not edit the verifier" constraint, applying minimal fixes, and recognizing a passing state. These are behaviors instilled by post-training for agentic use, not by scale.

### 11.2 The capability frontier is orchestration, not code

Every model can write a correct stats CLI, an LRU cache, or an optimized GEMM. The frontier is *multi-step orchestration under isolation*: fan-out (53%), tournaments (70%), and critic-gated loops (73%). These require a model to hold and honor a contract across files and sub-agents it cannot all see at once. This is exactly where a scientific-software agent must operate—coordinating adapters, harnesses, and benchmarks—so it is the most important gap to close.

### 11.3 Implications for delegating real work

For a research group, the practical recommendation is **gpt-oss:20b as the default local agent**: joint-best accuracy at the lowest cost, making it ideal for the many single-shot and iterate-until-green chores that dominate day-to-day software work. **devstral2:24b** is the strongest choice when a task leans on oracle-style differential repair (it alone solved the stable-softmax problem). Reserve the harder fan-out/tournament orchestrations for a frontier model until local models improve on those patterns.

### 11.4 Threats to validity

(i) The extraction and tool-use harnesses measure different things and must not be pooled. (ii) Per-turn and per-loop timeouts bound the search; a model near a solution at the budget edge is scored a failure—realistic for a cost-bounded agent but not a pure capability ceiling. (iii) Verifier design encodes judgment; we mitigate this by asserting every seed fails before looping, so no problem is pre-solved. (iv) Quantization (Q4-class) may modestly depress all models relative to full precision.

## 12 Conclusion

Across 150 verification-gated agent runs, local open models are already *competent* at the core agentic loop—one-shot generation, iterate-until-green, measurement-driven optimization, and reflection all pass at ≥93%. The remaining frontier is *orchestration*: fan-out delegation, tournament candidate supply, and critic-gated convergence, which fail 27–47% of the time even for the best local models. Encouragingly, the leaders on this frontier are small—**gpt-oss:20b** and **devstral2:24b** at 27/30—and gpt-oss:20b achieves it at a fraction of the compute of every competitor, underscoring that *agentic discipline, not scale*, is the operative variable. For a group running its own hardware, that is good news: a capable autonomous coding agent fits comfortably on a single local accelerator today, and the path to closing the orchestration gap is a training-and-tooling problem, not a hardware one.

**Three lines to leave you with.**

1. A loop without a machine-checkable verifier is just a chatbot—so we scored only exit codes.
2. Small, well-tuned agents (gpt-oss:20b, 27/30 in 0.9 h) beat larger, undisciplined ones.
3. The gap is orchestration (fan-out, tournaments, critics)—the next thing to train for.

## A Problem Catalog

Table 9 maps each problem to its concrete task and verifier family, so the verdict matrix (Table 10) can be read against real deliverables. Tasks are drawn from HPC/scientific-software chores by design, so the benchmark rewards the competencies a lab actually needs from a coding agent.

Table 9: The 30-problem catalog (representative tasks per tier).

| P | Pattern | Task |
|---|---|---|
| P1 | one-shot | Stats CLI: stream numbers → count/min/max/mean/median/sample-stdev. |
| P2 | bundled-tests | LRU cache with correct eviction *plus* its own pytest suite. |
| P3 | iterate-green | Roman numerals with subtractive notation; pass a hidden test file. |
| P4 | oracle | Numerically-stable softmax vs. a high-precision reference. |
| P5 | measurement | Optimize a GEMM kernel to a GFLOP/s target. |
| P6 | gen+critic | Reproducible FP reduction; critic gates on determinism. |
| P7 | fan-out | Three parse backends (csv/kv/json) + integrator dispatch. |
| P8 | reflection | ODE integrator convergence; persist a root-cause lesson. |

| P | Pattern | Task |
|---|---|---|
| P9 | tournament | Best-of-4 Levenshtein by correctness×speed. |
| P10 | capstone | Prime sieve: tournament + critic + benchmark. |
| P11 | iterate-green | Balanced-parentheses validator (stack). |
| P12 | oracle | Base-$N$ conversion round-trip vs. `int(s,base)`. |
| P13 | iterate-green | Mini-JSON parser (no `json`/`eval`). |
| P14–16 | mixed | Tier-2 realizations of one-shot / bundled / gen+critic. |
| P17 | fan-out | Multi-backend orchestration (Tier-2 fan-out). |
| P18 | reflection | Welford online variance (numerically stable). |
| P19 | tournament | Tier-2 best-of-$N$ optimization. |
| P20 | capstone | Tier-2 compositional system. |
| P21–23 | mixed | Tier-3 oracle / measurement (e.g. prime-count sieve). |
| P24 | gen+critic | Reproducible parallel reduction; `nchunks`-invariant. |
| P25 | fan-out | Tier-3 multi-worker decomposition. |
| P26 | reflection | Tier-3 lesson-persisting convergence. |
| P27 | tournament | Tier-3 best-of-$N$. |
| P28–29 | mixed | Tier-3 oracle / iterate-green. |
| P30 | capstone | Fastest-correct-sort capstone (tournament + bench). |

## B Full Per-Problem Matrix

Table 10 lists every verdict. Legend: ✓ pass, ✗ fail.

Table 10: Complete 5 × 30 verdict matrix with pattern labels.

| P | Pattern | gpt-oss | devstral2 | qwen3.6:35b | qwen3.6:27b | gemma4 |
|---|---|---|---|---|---|---|
| P1 | `oneshot` | ✓ | ✓ | ✓ | ✓ | ✓ |
| P2 | `oneshot` | ✓ | ✓ | ✓ | ✓ | ✓ |
| P3 | `iterate_green` | ✓ | ✓ | ✓ | ✓ | ✓ |
| P4 | `oracle` | ✗ | ✓ | ✗ | ✗ | ✗ |
| P5 | `measurement` | ✓ | ✓ | ✓ | ✓ | ✓ |
| P6 | `gen_critic` | ✓ | ✓ | ✓ | ✓ | ✓ |
| P7 | `fanout_p7` | ✓ | ✗ | ✗ | ✓ | ✓ |
| P8 | `reflection` | ✓ | ✓ | ✓ | ✓ | ✓ |
| P9 | `tournament_p9` | ✓ | ✓ | ✗ | ✗ | ✗ |
| P10 | `tournament_p10` | ✓ | ✓ | ✗ | ✗ | ✗ |

| P | Pattern | gpt-oss | devstral2 | qwen35 | qwen27 | gemma4 |
|---|---|---|---|---|---|---|
| P11 | iterate_green | ✓ | ✓ | ✓ | ✓ | ✓ |
| P12 | oracle | ✓ | ✓ | ✓ | ✓ | ✓ |
| P13 | iterate_green | ✓ | ✓ | ✓ | ✓ | ✓ |
| P14 | oracle | ✓ | ✓ | ✓ | ✓ | ✓ |
| P15 | measurement | ✓ | ✓ | ✓ | ✓ | ✓ |
| P16 | gen_critic | ✓ | ✓ | ✓ | ✓ | ✓ |
| P17 | fanout_p17 | ✗ | ✗ | ✓ | ✗ | ✗ |
| P18 | reflection | ✓ | ✓ | ✓ | ✓ | ✓ |
| P19 | tournament_p19 | ✓ | ✓ | ✓ | ✓ | ✗ |
| P20 | iterate_green | ✓ | ✓ | ✓ | ✓ | ✓ |
| P21 | oracle | ✓ | ✓ | ✓ | ✓ | ✓ |
| P22 | iterate_green | ✓ | ✓ | ✓ | ✓ | ✓ |
| P23 | measurement | ✓ | ✓ | ✓ | ✓ | ✓ |
| P24 | gen_critic | ✗ | ✓ | ✗ | ✗ | ✗ |
| P25 | fanout_p25 | ✓ | ✓ | ✓ | ✓ | ✗ |
| P26 | reflection | ✓ | ✓ | ✓ | ✓ | ✗ |
| P27 | tournament_p27 | ✓ | ✓ | ✓ | ✓ | ✗ |
| P28 | oracle | ✓ | ✓ | ✓ | ✓ | ✓ |
| P29 | iterate_green | ✓ | ✗ | ✓ | ✓ | ✓ |
| P30 | tournament_p30 | ✓ | ✓ | ✓ | ✓ | ✗ |

# C Reproducibility and Methodology Details

## C.1 Invocation

Each agent turn was a single non-interactive Pi call of the form:

```
pi -p --provider ollama-local --model <M> --no-session -na \
   --thinking off "<task or repair prompt>"
```

with file and `bash` tools enabled and a 360 s per-turn timeout. Iterate/oracle loops re-invoked Pi with the latest verifier output appended to the prompt; tournaments spawned four independent candidate directories, each its own fresh Pi context.

## C.2 Verifier families

The curriculum uses five verifier families, all yielding a Boolean exit code:

- **pytest** (iterate-green): the agent must make a provided, never-edited test file green (Roman numerals, balanced parens, mini-JSON parser).
- **differential oracle** (softmax, base-$N$): a slow-but-correct reference; the agent's fast version must match to tolerance over random inputs.
- **measurement** (GEMM, prime-count): a benchmark prints a throughput number; the loop must exceed a target or plateau.

- **convergence** (ODE integrator, Welford variance): a numerical convergence check must pass.
- **tournament scorer** (levenshtein, sieve, sort): fitness = correct ? speed:0; a champion must score $> 0$.

### C.3 Contamination guard

Before any sweep, `assert_seeds_fail.sh` confirms every seed *fails* its verifier. If a seed passed with zero model work, the arena would be pre-solved and any subsequent "pass" a phantom. This guard licenses the exit-code-only verdict discipline.

### C.4 Data provenance

The 30-problem tool-use matrix (150 runs) was produced by `matrix_resume.sh` on `m1acbook-2` and parsed from `MATRIX_FULL.log` (per-run verdict, attempt count, elapsed seconds, pattern). Cross-fleet extraction results were aggregated from `runs_extract/*/SUMMARY.txt`. All figures are generated directly from `matrix_records.json`.

### C.5 Hardware and serving

Models were served by Ollama with Q4-class quantization, one model resident at a time, at a 32K context window (required to fit Pi's system prompt plus tool schemas). Verifiers executed under a NumPy/Numba/pytest stack, which is why measurement problems achieved realistic throughput (Numba-JIT and vectorized NumPy paths).

## D Threats, Limitations, and Future Work

**Comparability.** The 30-problem tool-use scores and the 10-problem extraction scores measure different regimes and are never pooled. Tool-use is strictly harder (cross-file contracts, bash, incremental edits); extraction regenerates a single file per turn.

**Budgets as capability proxy.** Timeouts and attempt caps mean a model one edit from success at the budget edge is scored a failure. Faithful to a cost-bounded production agent, but it understates a pure capability ceiling. Future work: re-run the fragile patterns with 2–4× budgets to separate *capability* from *efficiency*.

**Quantization.** All models ran at Q4-class quantization; a full-precision re-run of the leaders would quantify the quantization tax on agentic behavior.

**Coverage gaps.** `m3acbook`'s 55-model LM Studio library was never swept, and `studio-ts` crashed. Completing an LM Studio tool-use sweep on `m3acbook` would add a large, directly-comparable set of local models.

**Prompt sensitivity.** Repair prompts were fixed strings. Some fan-out failures may be recoverable with a more explicit cross-file interface contract in the prompt—an inexpensive intervention worth testing before concluding a model *cannot* orchestrate.